

Stoquify Agent Runtime Proposal Explanation

Date: 2026-07-21

Refined Prompt

Act as a principal product strategist, enterprise software architect, AI agent systems architect, security/RBAC architect, and SaaS growth advisor for Stoquify.

Explain in detail the following proposal for Stoquify's agent and skill system:

1. Build one shared agent runtime inside Stoquify, not one framework per module.
2. Use TypeScript-native building blocks first: Vercel AI SDK or similar TypeScript-native tool loops, Model Context Protocol tools, Inngest/Trigger.dev-style durable execution, and Langfuse/Helicone/LiteLLM-style observability and model routing.
3. Keep domain skills Stoquify-native because the moat is tenant-scoped evidence, RBAC, OHADA context, stock-to-cash-to-close continuity, and controlled workflow execution.
4. Launch three user-facing agents first: Command Agent, Cash/Reconciliation Agent, and Inventory/Replenishment Agent. Treat Exception and Action Orchestrator as platform infrastructure.
5. Do not give any agent direct Prisma write access, direct ledger posting, direct statutory filing, or permission-changing authority.

Explain what this means technically, strategically, commercially, and operationally. Show where this proposal would place Stoquify in the market, what kind of product Stoquify becomes if it follows this plan, what risks it avoids, what competitive advantage it creates, and how to execute the plan step by step over 30, 60, 90 days and beyond.

Be concrete. Tie every recommendation to Stoquify's current architecture: Next.js, Prisma, services, RBAC, tenant scope, inventory, POS, cash, reconciliation, accounting close, compliance, payroll, OHADA evidence, and workflow assurance. Do not recommend generic AI automation. Prioritize the least code, highest leverage, safest path.

Detailed Explanation

This proposal moves Stoquify from being "a business management SaaS with dashboards" into an evidence-backed operating copilot for African and OHADA-oriented SMBs.

That is a much stronger position.

Instead of adding random AI chat features, Stoquify would become a system that says:

Here is what happened in your business, here is what matters today, here is the evidence, here is the risk, here is the recommended action, and here is the controlled workflow to resolve it.

That is powerful because Stoquify already has the hard part: POS, inventory, purchasing, payments, reconciliation, accounting close, payroll, compliance, RBAC, audit logs, workflow assurance, and evidence

trails. The agent system should sit on top of that truth. It should not replace it.

What Each Point Means

1. Build one shared agent runtime

This means Stoquify should have one internal agent control plane that all agents use.

Not:

- one framework for inventory,
- another framework for payroll,
- another for reconciliation,
- another for compliance.

That would create chaos.

Instead, Stoquify should have one shared runtime responsible for:

- agent definitions,
- skill versions,
- tool permissions,
- run history,
- approvals,
- evidence links,
- cost tracking,
- error states,
- audit records,
- feedback,
- incident handling.

This gives Stoquify professional control. Every agent follows the same safety rules.

2. Use TypeScript-native building blocks

Stoquify is already a Next.js, TypeScript, and Prisma system. So the agent layer should fit that architecture.

The best stack direction is:

- Vercel AI SDK or similar: agent/tool loops, structured outputs, streaming UI.
- MCP tools: expose Stoquify services as controlled tools.
- Inngest or Trigger.dev: durable workflows that can pause, wait for approval, retry, and resume.
- Langfuse, Helicone, or LiteLLM: traces, model costs, prompt versions, evaluations, routing.

This avoids rewriting the platform in Python or importing a giant external agent framework that does not fit the app.

3. Keep domain skills Stoquify-native

This is the most important strategic point.

Stoquify's moat is not "AI can chat."

The moat is:

- tenant-scoped business truth,
- RBAC and permissions,
- OHADA-aware workflows,
- stock-to-cash-to-close continuity,
- evidence-backed recommendations,
- accounting and compliance control,
- low-connectivity/offline POS awareness,
- trusted workflows for managers, accountants, and owners.

So the reusable skills should be Stoquify-specific:

- Trusted Context Resolver
- Permission and Entitlement Guard
- Evidence-Grounded Retrieval
- Cash Exception Triage
- Inventory Risk and Replenishment
- Reconciliation Match Suggestion
- Close Blocker Navigator
- Country Pack Provenance Resolver
- Report Trust Export Certifier

External frameworks give plumbing. Stoquify's own skills create defensibility.

4. Launch three user-facing agents first

The first three agents should be chosen because they create daily value.

The Command Agent becomes the main operating assistant. It tells the owner, manager, accountant, or cashier what matters today.

The Cash/Reconciliation Agent attacks one of the biggest pain points: "Where is the money, what does not match, and what needs action?"

The Inventory/Replenishment Agent attacks another daily pain point: "What is running out, what is overstocked, what moved strangely, and what should I reorder?"

The Exception and Action Orchestrator should not be a visible chatbot. It should be infrastructure that routes approvals, drafts actions, records evidence, and resumes workflows.

5. Do not give agents dangerous direct authority

This protects Stoquify from becoming unsafe.

Agents should not:

- write directly to Prisma,
- post ledger entries,
- file statutory reports,
- change permissions,
- approve payroll,
- change roles,
- certify close packs,
- write off stock,
- adjust cash without approval.

Agents can recommend, explain, draft, and route. Actual risky actions must go through Stoquify services, permissions, fresh-auth, approval, idempotency, and audit.

That is how Stoquify becomes trusted rather than flashy.

Where This Places Stoquify

If executed well, this places Stoquify above ordinary SMB SaaS.

Most SMB tools are record-keeping systems. They help users enter transactions and later view reports.

Stoquify can become an operating intelligence layer:

- not just POS, but cash truth;
- not just inventory, but stock risk and replenishment action;
- not just accounting, but close assurance;
- not just payroll, but readiness and compliance evidence;
- not just dashboards, but daily prioritized action;
- not just AI chat, but controlled evidence-backed execution.

That positions Stoquify closer to an enterprise operating system for SMBs than a simple stock or POS app.

How To Execute The Plan

First 30 days

- Define the agent runtime schema.
- Create `AgentRun`, `AgentStep`, `AgentEvidenceLink`, `AgentApproval`, `AgentFeedback`, and `AgentCost`.

- Wrap existing service read models as tools.
- Build Trusted Context, Permission Guard, Evidence Retrieval, Redaction, Freshness, and Run State skills.
- Ship read-only Command Agent internally.

Days 31-60

- Add Cash/Reconciliation Agent in read-and-draft mode.
- Add Inventory/Replenishment Agent in read-and-draft mode.
- Add approval workflows for recommendations.
- Track acceptance, rejection, stale answers, wrong answers, and time saved.

Days 61-90

- Allow only low-risk controlled service actions.
- Add idempotent tool execution.
- Add Langfuse/Helicone-style tracing and cost tracking.
- Add Platform Assurance Agent to watch agent failures, cost, trust, and policy breaches.
- Keep ledger, statutory, payroll, permission, and close certification actions human-owned.

After 90 days

- Add Purchasing/AP Agent.
- Add Close and Compliance Agent.
- Add Payroll Readiness Agent.
- Add Customer Success/Adoption Agent.
- Start turning accepted agent recommendations into product analytics and workflow intelligence.

Final Destination

The destination is clear: Stoquify becomes the trusted daily command system for SMB operators, accountants, and managers.

It becomes not just "software they use," but the system that tells them what needs attention and helps them resolve it safely.